

Leaper Seeker Experiment Plan

Recommendation

Reopen recurrent memory as a controlled experiment. Preserve the existing walking policy and add a small, trainable memory branch. Start with one LSTM layer of 64 units, approximately 73,000 total trainable parameters, and compare it with a similarly sized feedforward model, a finite-history model, and a GRU. Keep the stationary-target environment and reward fixed for these architecture comparisons.

The previous experiments justify rejecting those particular LSTM configurations. They do not establish that learned memory cannot help seeking. Conversely, adding an LSTM does not guarantee that the model will build a useful spatial map, explore efficiently, or stop colliding. Those are separate hypotheses requiring separate measurements.

This plan contains 15 experimental entries: two diagnostic prerequisites and 13 training comparisons. Run them in stages, with inexpensive screens and mandatory longer budgets for healthy recurrent candidates. The first decision is whether memory improves discovery and successful arrival over a continued MLP, not whether a large network can eventually produce an attractive replay.

The immediate priority is broader than search coverage. A completed ablation on six PPO_34/35 models shows only a small benefit from hiding frontier inputs after detection. Its detailed endings also show that collision and freeze termination account for most failures in that sample. Measure discovery, pursuit, and physical failures independently before assigning the whole plateau to memory.^{1, 3}

1. Current system and evidence

Task and observation

The current task is a partially observable navigation problem: find a stationary, initially potentially hidden target, then reach it. It is not yet adversarial hide-and-seek. The seeker arena is approximately 62.5 units wide with six randomized obstacles, a 1,000-step cap, 270-degree target visibility, and a 28-unit sensing range. Physical movement is forward-only, up to 0.375 units per step, with turns up to 9 degrees per step. Collision checks include the body and leg samples.¹

The PPO_35 observation has 31 numbers. The following groups are actual model inputs, not instructions written in natural language.

Indices	Contents	What the model can infer
0-4	Target visible; time since last sighting; two world-frame direction components; normalized distance to last-seen target	Whether there is a target observation and where its remembered location lies
5-9	Sine/cosine of yaw; previous collision; previous two policy actions	Orientation and recent control context
10-25	Sixteen obstacle/wall range measurements	Current local clearance
26-30	Frontier direction x/z, normalized distance, uncleared-region fraction, validity flag	One suggested region to investigate

Before discovery, target direction is zero and distance uses its unknown sentinel. The target's last-seen position is maintained by environment code after discovery. The age input saturates after 120 steps, but the remembered

1. Leaper environment, trainer, ledger, and archived PPO_27-35 records; see Sources 1.

3. Saved paired frontier-mask evaluation, note_gate_full.json; see Sources 3.

position does **not** expire then. For the stationary target, this memory is reliable even when the target leaves view.¹

The external coverage representation records which of 441 grid-cell centers have been visually checked. It is neither a full occupancy map nor a learned belief distribution. Its updates do not inspect the real target location, but do use the simulator's visibility geometry. The actor receives five compressed frontier features, not the grid. A cell chosen as the nearest edge of the largest uncleared region can be an unsuitable walking destination, including a point inside an obstacle. Knowing where unobserved space lies is different from knowing a safe vantage point from which to inspect it.¹

Model and optimizer

The current PPO actor and critic have separate 31 -> 64 -> 64 Tanh networks. The actor produces two Gaussian action means and learns two log standard deviations; the critic produces one scalar value estimate. Total trainable parameters: **12,613**. There is no recurrent state, learned search/pursuit selector, or separate search-action head. The actor and critic are already two different prediction heads with different training objectives.¹

Policy actions are clipped to [-1, 1]; physical throttle is (action[0] + 1) / 2. Thus a zero policy throttle means half-speed forward movement. This is not a final-layer Tanh policy. Replacing Gaussian clipping with a squashed distribution would require consistent log-probability calculations, so it should not be bundled into an activation experiment.

PPO_35 uses learning rate 0.00015, eight environments, 1,024 steps per environment, 8,192 transitions per rollout, batch size 256, ten optimization epochs, discount 0.995, GAE lambda 0.95, and entropy coefficient 0.01. PPO defaults supply clip range 0.2, value coefficient 0.5, and gradient norm limit 0.5. The local packages are Stable-Baselines3 and sb3-contrib 2.9.0. These are starting settings, not experimentally established optima for the proposed recurrent architecture.¹

1. Leaper environment, trainer, ledger, and archived PPO_27-35 records; see Sources 1.

PPO_27 through PPO_35

Run	Important change	Deterministic result	Interpretation
27	Hidden target; first seeker	64%	Seeking works to a useful initial degree; repeated sight reward allowed an exploit
28	Slower movement; sight reward paid only once	71%	Better score, but many stopped actions
29	Normalized throttle, freeze rule, scan reward	69%	Stronger movement contract; current seeker fallback
30	Fresh, one-layer LSTM, 256 units	53% at 200k; 39% at 500k	Two different training seeds/runs, not one policy degrading during an extension
31	Two LSTM layers, larger exploration rewards, fewer epochs	About 20% at last small evaluation; stopped near 164k	Confounded, incomplete experiment; termination cause unproven
32	Sixteen local coverage features; MLP transfer	67.3% across three seeds	Added information did not improve performance
33	Five global frontier features; zero new input weights	70.7% across three seeds	Little measured use of frontier features
34	Initialize direction weights at 0.5 times target-direction weights	70.0%; first detection 88.0%	Better discovery, weaker conversion from discovery to arrival
35	Initialization multiplier 0.25; train 500k	72.3%; first detection 85.7%	Small overall improvement; existing promotion gate unmet

The 0, 0.25, and 0.5 values describe **initial weights**. They are not permanent rules or fixed levels of trust in a frontier suggestion. The transferred weights remain trainable. PPO_35 also changes training duration relative to PPO_34, so that comparison cannot isolate the initialization multiplier.¹

PPO_30's original network has approximately 623,000 parameters; PPO_31 has approximately 1.675 million. Both discard the established MLP policy rather than preserve it through an identity-preserving extension. PPO_31 also changes rewards and update epochs. Its log ends without a traceback: the ledger's out-of-memory explanation is plausible speculation, not demonstrated evidence. These details weaken a general conclusion that recurrent memory is unsuitable.

Completed frontier-mask ablation

The existing `rl_artifacts/note_gate/note_gate_full.json` already evaluates normal inputs against masking all five frontier channels after first detection, on the same 100 maze seeds for each model. The invalid-note vector is [0, 0, 1, 0, 0], meaning no direction, sentinel distance, zero region fraction, and invalid. It removes a spatial suggestion, not a text instruction.³

¹. Leaper environment, trainer, ledger, and archived PPO_27-35 records; see Sources 1.

³. Saved paired frontier-mask evaluation, `note_gate_full.json`; see Sources 3.

Model	Normal success	Masked after detection	Change
PPO_34 s1	65%	68%	+3 points
PPO_34 s2	73%	75%	+2 points
PPO_34 s3	72%	74%	+2 points
PPO_35 s1	71%	73%	+2 points
PPO_35 s2	72%	74%	+2 points
PPO_35 s3	74%	72%	-2 points
Pooled model-maze outcomes	71.17%	72.67%	+1.50 points

First detection is unchanged, as expected for an intervention applied afterward. The normal-condition 600 outcomes contain 427 arrivals, 116 stuck endings, 33 frozen endings, and only 24 genuine time-limit endings. Of 173 failures, 149 terminate early through stuck/freeze rules. The effect of masking is small and inconsistent across models; it does not explain the plateau by itself.

These are 600 model-maze outcomes, not 600 independent maze layouts. A post-training mask is also a distribution shift; even a meaningful mask effect would not prove that training with a learned selector will produce the same result. The saved evaluation took approximately 285 seconds historically. Repeating this entire test is unnecessary unless its contracts fail audit.

2. Reward design and the document's LSTM advice

Why +25 does not automatically teach good seeking

The current reward already pays exploration bonuses only **before** first discovery. Afterward, the exploration term is zero, even if the frontier observation remains present. First sight pays +0.5 once. Pursuit pays $0.2 * (\text{previous distance} - \text{current distance})$, clipped by movement speed. Reaching the target pays +25. Therefore, persistent post-discovery wandering cannot simply be explained as collecting ongoing exploration bonuses.¹

Before discovery, the policy receives +0.01 for a new visited cell, +0.002 for a new cell-and-heading view, and +0.01 for a previously unscanned heading, with heading novelty capped. There is also positive-only hidden-distance progress shaping at scale 0.1. Costs include -0.002 per step, -0.18 per collision, and -0.04 for idling after its grace period. Forty consecutive collision steps without displacement trigger a terminal -10 stuck failure; sixty steps without displacement trigger a terminal -10 frozen failure. The detailed flags must remain separate in evaluation.

A large terminal reward matters only through experienced trajectories and the gradients learned from them. With discount 0.995, a +25 reward 200 steps away contributes approximately 9.17 to today's discounted return; at 500 steps it contributes about 2.04. Collision costs, limited successful experience, inaccurate values, and local action distributions can still favor poor behavior. The GAE trace factor is $0.995 * 0.95 = 0.94525$; its geometric scale is about 18 steps. That is not a hard learning horizon, but it makes bootstrapped value quality important for long searches.

Do not simply increase all novelty rewards or add a reward for moving. Movement can be useless, and more exploration reward previously changed the optimization problem in an undesirable direction. Experiment 12 tests bounded reward for newly observed space. Experiment 15 removes privileged pre-discovery distance shaping to learn whether it is helping or distracting. Neither claims policy-invariant shaping: the established invariance result

1. Leaper environment, trainer, ledger, and archived PPO_27-35 records; see Sources 1.

requires the specific form $\gamma * \Phi(\text{next state}) - \Phi(\text{state})$ and appropriate terminal handling, which the existing collection of bonuses does not satisfy.⁸

The hidden-distance term is privileged **training supervision**, not direct target-coordinate input to the actor. Reward must remain excluded from the proposed recurrent input; otherwise that shaping signal becomes an additional temporal observation of the hidden target.

Why the supplied PDF favors maintained memory

The supplied *Leaper_Memory_Explainer_and_Answers.pdf* recommends retaining useful behavior, adding explicit search information, and proving the stationary phase before increasing task difficulty. This is a reasonable engineering recommendation: a deterministic map is easier to inspect than a latent recurrent state, and rebuilding walking, obstacle avoidance, and spatial memory together creates a harder optimization problem.²

Its advice should be treated as a preferred starting strategy rather than an architectural impossibility result. A stationary hidden target still creates partial observability: identical local sensor readings can occur after very different search histories. Remembering where the target was seen solves one memory problem; remembering which routes and viewpoints have already been tried solves another. A five-number frontier summary is not necessarily a sufficient state representation.

The proposed compromise follows the document's preservation principle while testing recurrence fairly: keep the transferred walker, retain existing notes initially, and learn a small temporal correction. Remove one external memory component only after that branch demonstrates an advantage. Progress to moving targets after stationary seeking passes its gate.

3. Research implications

Recurrent control. Ni, Eysenbach, and Salakhutdinov show that tuned recurrent model-free RL can be competitive across many partially observable tasks. Their work highlights separate actor/critic recurrent networks, context length, and algorithm choice; their off-policy comparisons also outperform recurrent PPO in several settings. This supports a serious recurrent baseline, not a claim that LSTM plus PPO will solve Leaper. Test sequence length and a GRU alongside LSTM; defer recurrent TD3/SAC until this smaller comparison reveals a learning limitation.⁴

Spatial navigation. Active Neural SLAM separates spatial mapping, goal selection, planning, and local control. Classical frontier exploration chooses accessible observation positions near the boundary between known free and unknown space. These approaches suggest improving the meaning of Leaper's frontier feature rather than assuming an MLP can transform any uncleared cell into a useful walking goal. Their environments and sensing pipelines differ from Leaper, so their reported success rates are not transferable.^{5,6}

Learning useful memory. Ye and colleagues' ObjectGoal navigation work provides evidence for recurrent representations trained with auxiliary tasks and exploration. This motivates a modest coverage-prediction objective: the latent state should encode something testable about previously observed space. It does not imply that a perfect predicted map guarantees good action selection.⁷

Activation functions. Tanh hidden layers are a legitimate choice in continuous-control PPO. The large on-policy study by Andrychowicz and colleagues found useful shallow Tanh configurations and substantial interaction among implementation choices. ReLU is worth a controlled test, but changing activations is a lower-priority hypothesis than memory, spatial observability, and failure classification. Keep standard sigmoid/Tanh gates inside LSTM; changing those gates creates a different recurrent architecture.⁹

Adversarial hide-and-seek. Baker and colleagues study competing agents and emergent curricula. That is relevant to the eventual game, but self-play introduces a changing opponent distribution and a much larger training problem. A stationary-target seeker is a clearer current benchmark. Add a moving target before adding a learning hider.¹⁰

2. Supplied *Leaper_Memory_Explainer_and_Answers.pdf*, pp. 27-28; see Sources 2.

4. Ni et al., Recurrent Model-Free RL, ICML 2022.

5. Chaplot et al., Active Neural SLAM, ICLR 2020.

6. Yamauchi, Frontier-Based Autonomous Exploration, CIRA 1997.

7. Ye et al., Auxiliary Tasks and Exploration, ICCV 2021.

8. Ng et al., Policy Invariance Under Reward Transformations, ICML 1999.

9. Andrychowicz et al., What Matters In On-Policy RL?, ICLR 2021.

10. Baker et al., Emergent Tool Use From Multi-Agent Autocurricula, ICLR 2020.

Experimental reliability. Small evaluation sets and a single training seed can make RL rankings unstable. Use shared maze seeds, multiple training seeds for finalists, uncertainty intervals, and a reserved final exam. Three training seeds are a practical minimum here, not strong evidence of universal robustness.¹¹

4. Proposed model family

Maintained memory, learned memory, and heads

Maintained memory is explicit environment state: last-seen target coordinates, their age, and accumulated coverage. State augmentation exposes a summary of that state in the observation. The existing MLP itself remembers nothing between calls.

An LSTM maintains hidden state h and cell state c , updated every decision. They are learned numerical summaries of history, not named map cells. A GRU maintains a hidden state with a simpler gating structure. Both are recurrent neural networks; a vanilla Tanh RNN is not a prerequisite and would be a lower-priority comparison for long histories.

A "node" usually means a scalar hidden unit in a network. The earlier "note" means a group of observation features. Neither implies another AI agent. A head is an output mapping, such as the actor's action means, the critic's value, or an auxiliary coverage prediction. The proposed main model has separate actor and critic recurrent branches; it does not have competing search and pursuit action heads or a hand-coded handoff.

Preserve the current policy exactly at initialization

Select ppo_35_frontier_seed025_500k_s1/leaper_ppo.zip as the common donor before comparing outcomes. This is not the highest-scoring PPO_35 seed. It deliberately uses the latest input interface; PPO_29 remains a historical benchmark. Do not mix donor checkpoints within the main comparison.

Append three odometry features: actual world-frame displacement x/z divided by 0.375, and wrapped actual yaw change divided by the maximum turn. Reset all three to zero. These are measured motion outcomes, not commanded actions; collisions can make them different. They make integration of spatial history better posed without exposing target coordinates or the unseen obstacle map. All main candidates, including controls, receive the same 34 inputs. Zero-initialize the corresponding columns in the transferred MLP.

For each actor/critic branch, let $\text{base}(o)$ be the transferred final 64-dimensional latent vector. Add a trainable residual branch:

```
34 inputs -> transferred 64-Tanh -> 64-Tanh -----+
                                                    + -> existing output head
34 inputs -> one-layer LSTM(64) -> Linear(64, 64) -----+
```

Initialize the last projection's weights and bias to zero. This preserves the donor's means, value, and log standard deviations at step zero. All base weights remain trainable. Initially only the projection receives a useful branch gradient; verify that recurrent gradients become nonzero after the projection starts learning. This is an architectural recommendation, not an existing trainer feature.

Use independent actor and critic memory parameters and states, one layer, FP32, no dropout, and no extra learned gating network. Use standard PyTorch recurrent initialization initially; record the actual initialization and its seed. A later initialization change would be another experiment. Never reset hidden state merely because the target is detected.

Candidate	Branch per actor/critic, beyond transferred base	Total parameters
Continued MLP	No additional branch; 34-input base	12,997
Capacity control	34 -> 128 -> 112 -> 64, Tanh; zero residual projection	73,637
Four-snapshot history	140 inputs -> 64 -> 64, Tanh; zero residual projection	47,685
LSTM-64	One recurrent layer; zero 64 -> 64 projection	72,517
GRU-64	One recurrent layer; zero 64 -> 64 projection	59,717
LSTM-128	One recurrent layer; zero 128 -> 64 projection	197,445
LSTM-64 + auxiliary task	LSTM-64 plus one 49-output linear head on each recurrent branch	78,887

Counts include actor, critic, biases, and the two Gaussian log standard deviations. They were checked by instantiating the specified networks. The LSTM-64's actor alone has 36,260 parameters. Auxiliary heads are used only during training and are omitted from deployment.

The capacity control is within approximately 1.5% of LSTM-64's parameter count. If it performs equally well, additional function capacity may explain the gain. The history control takes observations at lags 0, 1, 8, and 32 plus four validity bits: $4 * 34 + 4 = 140$ inputs. Missing history is zero-filled and masked; history never crosses episode boundaries.

5. Fifteen experiments

Budgets below are additional transitions from the common donor, rounded to whole 8,192-transition rollouts. They include the initial screen. They are minimum decision budgets for functioning implementations, not proofs of convergence. "500k" means 507,904 collected transitions; "250k" means 253,952; "1M" means 1,007,616.

ID	Experiment and comparison	Minimum decision budget
01	Runtime, throughput, observation and recurrent-state audit	No scientific training result
02	Audit completed mask test; decompose failures	Evaluation only
03	Continued MLP with odometry; common control	250k; match finalist budgets
04	Matched-capacity feedforward residual vs 03 and 06	500k
05	Four-snapshot history vs 03 and 06	250k; extend if shortlisted
06	One-layer LSTM-64 residual vs 03/04/05	500k
07	GRU-64 residual vs 06	500k
08	LSTM-128 residual vs 06 at equal steps	1M; also extend 06 comparator
09	LSTM-64 training context 256 vs 128	500k
10	Remove frontier features from a qualified 06 policy	250k more; paired intact continuation

ID	Experiment and comparison	Minimum decision budget
11	LSTM-64 with coverage auxiliary loss vs 06	500k
12	Newly visible space reward vs 03	250k
13	Reachable observation viewpoint feature vs 03	250k
14	ReLU in new feedforward branch vs 04	500k
15	Remove hidden-distance reward before discovery vs 03	250k

01. Runtime and experimental contracts

Repair the runtime in an isolated environment, pin versions, and verify checkpoint loading. Benchmark the 34-input MLP and LSTM-64 on CPU and GPU with the same eight environments, rollout collection, optimizer work, and recording settings. Measure at least three rollouts after startup. Benchmark the full loop, not just a matrix multiply. Keep experimental timing separate from learning results.

Check donor-equivalent actions/values at initialization on recorded observations; actual-motion odometry; reset masks; time-limit bootstrapping; padding masks; hidden-state continuity; absence of cross-worker leakage; and nonzero gradients after the zero projection learns. A split-and-resume rollout must agree with continuous recurrent inference when state is restored. GPU and CPU actions should agree within a recorded numerical tolerance. No full sweep starts before these contracts pass.

02. Diagnostic baseline and failure taxonomy

Reuse the saved six-model mask results. Audit the mask vector and intervention timing on a small reproducible subset; verify pre-detection trajectories match, rather than relying only on matching aggregate detection rates. Log reached/stuck/frozen/truncated independently. If flags overlap, preserve both and define a consistent reporting precedence.

Stratify initially visible targets from actual search episodes, then classify failures before versus after first detection. For pursuit, measure time to arrival and failures, not only mean chase time among successes. Correct revisit metrics to count returning after leaving a cell; consecutive steps dwelling in one cell are not separate revisits. On recurrent candidates later, reset hidden state at predeclared intervals as an evaluation-only dependence probe. A performance drop shows dependence on memory, not that its contents form a correct map.

03. Continued MLP control

Continue the chosen donor with the common 34-input interface and otherwise unchanged rewards and PPO settings. This tests what more training and actual odometry can achieve without recurrence. It is the direct comparator for reward/feature experiments and the simplest architecture control. Extend it to every budget used for a finalist; comparing a 1M recurrent model only with a 250k MLP would confound training duration.

04. Matched-capacity feedforward residual

Use the 73,637-parameter branch defined above, with the same transfer, inputs, output heads, learning rate, and optimizer epochs as 06. It receives only the current observation. This separates useful temporal state from simply adding a larger nonlinear function. A win over 03 with no gap to 06 would argue against making recurrence the next default.

05. Finite-history control

Use the four snapshots at lags 0, 1, 8, and 32 with validity flags. This can estimate recent motion and recognize short loops without a recurrent optimizer. It cannot retain arbitrarily long search history. If it matches LSTM on arrival and discovery, retain it as the simpler baseline and test whether longer-horizon arenas actually need recurrence later.

06. Primary LSTM-64

Train the 72,517-parameter model with a maximum learning sequence length of 128, 32 preceding burn-in steps where available, and 256 valid learning tokens per optimizer minibatch. Hidden state persists throughout each episode at inference and rollout collection. Evaluate both aggregate success and discovery among initially hidden targets. This is the main test of learned temporal memory while preserving the established controller.

07. GRU-64

Change only the recurrent cell from LSTM to GRU, retaining one layer, 64 hidden units, sequence settings, and transfer. Its parameter count is lower by design; report that rather than claiming an exact capacity match. If performance is similar and runtime better, GRU is a reasonable game-facing candidate. Do not add a vanilla RNN experiment ahead of these stronger gated baselines.

08. LSTM-128

Increase only recurrent hidden size to 128, keeping one layer. Give it a 1M minimum decision budget, and extend LSTM-64 to 1M for comparison if this experiment is pursued. The hypothesis is insufficient representational capacity in 64 units. Run it only after 06 is operational and informative; parameter count alone is not evidence that scaling will help.

09. Longer training context

Keep LSTM-64 fixed and change maximum learning sequence length from 128 to 256. Keep valid tokens per update at 256 and burn-in at 32, which changes the number of sequences and temporal correlation inside a minibatch. Record this unavoidable coupling. Log the actual sequence-length distribution because early terminal failures can make nominally long chunks short. `n_steps=1024` is not a substitute for controlling the recurrent sampler.

10. Replace one external memory aid

Only after 06 demonstrates a credible gain, fork its selected checkpoint into an intact continuation and a frontier-masked continuation. Mask channels 26-30 throughout training and evaluation with the canonical invalid vector; keep target memory, odometry, and range sensors. Train both for another 250k. Initial masked degradation is expected distribution shift and is not the final result. This tests whether learned memory can substitute for the frontier summary; it does not test removal of every form of maintained memory at once.

11. Coverage auxiliary objective

Attach a linear 49-output prediction head to each LSTM-64 branch. Predict a 7-by-7 egocentric summary of previously observed area, with each label in $[0,1]$ equal to the cleared fraction of a fixed 9-by-9-unit bin relative to current pose. Define the footprint and out-of-world mask before training. Labels come from observation history and maintained coverage, never the target's hidden position.

Use masked mean binary cross-entropy with logits for these soft labels, averaged over bins, valid tokens, and both branches, with coefficient 0.05 added to the PPO objective. This is representation supervision, not environmental reward. Log prediction loss and PPO loss scales. A better map prediction with unchanged seeking is evidence that control, rather than memory representation alone, remains limiting.

12. Reward actual new visibility

From 03, replace the new-visited-cell and new-cell-heading bonuses with newly cleared cells / 441, only before first detection. Seed the reset coverage without reward. This contribution is bounded by 1.0 per episode because coverage is monotonic. Keep the small scan reward, first sight, pursuit, time, collision, idle, and terminal terms unchanged.

This tests whether the current visitation proxy encourages movement that reveals little useful space. It changes both the definition and scale of that one exploration component; record both. Use common task metrics for comparison because total return across different reward definitions is not directly comparable. Do not reward repeated visibility of the same space.

13. A reachable observation viewpoint

From 03, retain the same five frontier channels and uncleared-region selection, but point toward an observed-free vantage point from which the selected region might be inspected. Build observed-free/occupied evidence from past sensor geometry and actual odometry. Score candidates by approximate newly observable area minus path length and proximity risk; plan paths only through observed free space. Calibrate these geometric costs in fixed diagnostic examples before the run and keep them fixed.

This is the concrete meaning of "suggest a useful place to see more unexplored space." It means numerical direction/distance to a candidate viewpoint, not a command to walk there. The actor still chooses actions from reward. Do not query the complete hidden obstacle layout for an oracle route. For this planar simulator, the known footprint can reject unsafe candidates, but thin rays alone may leave clearance unknown. Mark an unavailable suggestion invalid. This experiment requires more implementation work than the other feature variants; no training-time estimate includes that engineering work.

14. Controlled activation test

Change Tanh to ReLU only in experiment 04's new feedforward branch, leaving the transferred base and output distribution unchanged. Keep architecture and initialization scheme otherwise fixed. This preserves step-zero behavior and makes the comparison interpretable. It answers whether ReLU helps the added branch; it does not answer whether replacing every hidden activation in a newly trained policy is better.

15. Remove pre-discovery privileged distance shaping

From 03, set pre-discovery BEST_PROGRESS_SCALE from 0.1 to zero. Keep the post-discovery signed progress term and all other rewards unchanged. This tests whether a reward correlated with an unseen target helps discovery or competes with general coverage behavior. It may reduce sample efficiency; do not reject it solely for lower training return. It is also a cleaner reference for future tasks where hidden target distance is unavailable during training.

6. Recurrent training and evaluation protocol

Sequence handling

Stable-Baselines3's recurrent extension provides LSTM PPO and state-aware prediction, but the proposed residual architecture, GRU, auxiliary task, and explicit context sampler need implementation. Existing CLI flags do not create these experiments. The installed rollout buffer splits sequences at environment/episode boundaries and minibatch boundaries; it does not make a chosen `n_steps` equal a fixed backpropagation horizon.¹²

Use contiguous chunks and preserve within-chunk ordering. Pad only where necessary and mask padded tokens out of policy, value, entropy, and auxiliary losses. The intended batch contains 256 valid learning tokens, excluding

12. Official SB3, sb3-contrib, PyTorch docs and local runtime inspection; see Sources 12.

burn-in and padding; report actual counts and loss normalization. Keep approximately the same ten passes over valid rollout tokens as the MLP rather than silently increasing optimizer work.

Initialize each chunk from its stored rollout state, run up to 32 preceding steps without gradient to refresh context, and detach before the learning segment. This is a practical approximation: cached hidden states can be stale after PPO updates. Log policy KL and instability, and document the approximation. Recomputing context from the complete episode prefix is a later alternative if state staleness is implicated. Never shuffle isolated recurrent timesteps.

Reset both h and c at episode starts independently for each worker. On a time-limit truncation, bootstrap using the terminal observation with its corresponding recurrent context before reset. Do not bootstrap true reached/stuck/frozen terminals. At deployment, retain state across frames and reset it on a new arena/episode. Memory carry at inference can span 1,000 steps even when gradients are truncated to 128 steps; that does not guarantee the network learns such long dependencies.

Fixed settings and stopping rules

Use the PPO_35 settings listed earlier for all initial comparisons, including **ten epochs for recurrent models**, overriding the current trainer's automatic five-epoch recurrent setting. This deliberately matches optimization exposure; record it explicitly. If repeated clipping, excessive KL, or nonfinite values indicate instability, stop and diagnose. Do not quietly tune only the best-looking candidate. A later five-epoch or lower-learning-rate test must have a matched recurrent control.

Each new implementation first gets a 24,576-transition engineering smoke test. Then use 106,496 transitions as an initial learning screen. Reject broken contracts, nonfinite learning, or unusable throughput; do not reject a healthy LSTM merely because it has not won at 100k. Respect the minimum budgets in the matrix. These are additional training steps: warm-starting from a 500k donor means a "500k experiment" has inherited substantial prior training.

Evaluate at approximately 50k-step intervals rounded to rollout boundaries. If a candidate is still improving at its floor, extend it and its comparator to the next common budget. A practical predeclared extension signal is at least +3 success points or +5 first-detection points across the last three development evaluations without worsening early terminal failures. Such a signal is a resource-allocation heuristic, not a statistical convergence test. A flat result means "no benefit within this budget," not "LSTM cannot work."

Evaluation sets, seeds, and metrics

Keep maze seeds 10,000-10,099 as the historical reference. They have been used repeatedly and are not an untouched generalization test. Establish a 200-maze development set, proposed seeds 20,000-20,199, and reserve 1,000 confirmation mazes, proposed seeds 60,000-60,999. Check previous artifacts for seed reuse before calling either set unseen. Freeze the confirmation set before training and do not use it for checkpoint selection.

Draw training seeds naturally and record them. For the controlled comparison, use the same naturally drawn seed within each experimental block, and three different blocks for finalists. This is a deliberate change from generating an unrelated seed for every variant. Paired seeds reduce some noise but do not make trajectories identical once policies differ. Initially screen one block; replicate the top two candidates plus their common control to three training seeds.

Report deterministic deployment evaluation separately from stochastic training results. Use development-selected checkpoints under the same selection rule and also report each final checkpoint. Avoid averaging only the best seed. Main metrics are success, first detection, success conditional on detection, pre/post-detection stuck/frozen/timeout rates, detection time with nondetections treated as censored failures, steps to arrival, unique visible coverage, return visits after exiting cells, and collision streaks. Include initially visible versus hidden-target strata.

Use paired maze outcomes for differences and bootstrap uncertainty, respecting shared maze IDs and training-seed blocks. Three training seeds leave substantial between-seed uncertainty; report per-seed values as well as pooled results. Do not treat repeated models on the same maze as independent environments. Auxiliary accuracy, hidden-state dependence, KL, entropy, value error, action saturation, gradient norms, and throughput are explanatory diagnostics, not substitutes for arrival success.¹¹

7. Hardware, duration, and execution order

Verified local resources

The machine has an **NVIDIA GeForce RTX 3050 Laptop GPU with 4,096 MiB VRAM**, an **AMD Ryzen 5 5600H**, **six physical cores/twelve logical processors**, and approximately **15.3 GiB system RAM**. The observed NVIDIA driver is 592.82. Its CUDA capability display does not mean that the installed PyTorch can use CUDA.

The repository's package directory contains **PyTorch 2.13.0+cpu**, and CUDA availability is false. The .venv Python launcher refers to a missing Python installation. The packages can currently be inspected through the bundled Python 3.12 runtime, but this is not a clean reproducible training launch. Create an isolated environment with a supported CUDA-enabled PyTorch build and compatible pinned SB3/contrib packages, then verify CPU/GPU equivalence. Do not modify archived models or replace packages in place merely to begin the sweep.¹²

The current eight environments use DummyVecEnv, so they execute sequentially in one process. Geometry, visibility, coverage, replay serialization, and evaluation can limit throughput regardless of GPU size. Small MLP PPO often benefits little from a GPU. Benchmark CPU threading and, separately, process-based environments before deciding that GPU utilization reflects training health. Use one training job at a time on 4 GB VRAM.¹²

Measured timing and planning estimates

Historical TensorBoard events for PPO_35 s1 span about 27.7 minutes through 507,904 transitions and report 303 transitions/second. PPO_30's 500k LSTM run spans about 93.9 minutes and reports 90 transitions/second. These are historical runs with different architectures and logging conditions; they do not establish how fast the new residual LSTM will train.

A current read-only CPU probe ran 4,096 deterministic policy-plus-environment steps in 6.85 seconds, about 598 steps/second with one PyTorch thread. It performed **zero training updates** and must not be used as a training-rate promise. The saved six-model frontier-mask test took approximately 4.8 minutes.

Additional steps	At 303 steps/sec	At 90 steps/sec	Interpretation
106,496	5.9 min	19.7 min	Initial screen
253,952	14.0 min	47.0 min	Minimum smaller-variant budget
507,904	27.9 min	94.1 min	Main recurrent decision budget
1,007,616	55.4 min	186.6 min	Larger model / extended comparison

These are arithmetic projections from historical rates, not measured new-model runtimes. New evaluation frequency, sequence burn-in, padding, auxiliary work, and CPU/GPU transfer can change them substantially. Measure full-loop wall time in experiment 01 and replace the estimates before queuing long runs.

All thirteen training entries screened once at 106,496 steps would total approximately 1.38 million additional transitions, or 1.3-4.3 hours at those illustrative rates, before extra diagnostics and new evaluation overhead. This is not the cost of completing the research: reaching every minimum budget, paired continuations, and multi-seed

¹¹. Agarwal et al., Statistical Precipice, NeurIPS 2021.

¹². Official SB3, sb3-contrib, PyTorch docs and local runtime inspection; see Sources 12.

confirmation requires several times more compute. Larger budgets cannot responsibly be collapsed into a set of five-minute experiments.

Staged execution

1. **Diagnostics:** complete 01 and 02. Reproduce checkpoint loading and failure classification, then establish full-loop runtime.
2. **Core memory comparison:** run 03, 04, 05, and 06 on one seed block. Screen at 100k, then complete their minimum budgets. Bring 03 to 500k for the central comparison.
3. **Targeted follow-ups:** test 07 and 09 if recurrent learning is healthy; test 11 if memory use is weak or coverage encoding poor; test 08 if capacity remains a plausible limitation. Test 12/13/15 against 03 to investigate search signals independently. Keep 14 lower priority.
4. **Memory substitution:** run 10 only if 06 qualifies. Continue the intact model for the same additional duration.
5. **Replication:** train the top two candidates and their control on two additional seed blocks at matched budgets. Freeze model and checkpoint-selection rules, then run the reserved 1,000-maze confirmation.

Every launch needs a unique artifact directory, recorded donor checksum, code revision plus working-tree patch, package versions, seed, input contract, reward configuration, parameter count, exact transitions, sampler settings, and wall time. Publish TensorBoard and replay links when launching training. Maintain a result row for stopped or failed runs as well as successful ones. This document specifies the experiments; it does not represent completed implementation or training.

8. Decision criteria and the moving-target phase

Retain the project's stationary promotion gate: mean success at least **78%** and first detection at least **90%** across three training seeds, followed by the reserved 1,000-maze confirmation. Require the confirmation means to meet the same thresholds; show intervals rather than implying certainty from a pass. Also require an improvement over the matched continued-MLP control, with per-seed results visible and no material increase in stuck/frozen outcomes. Predeclare a three-percentage-point increase in early terminal failures as a review trigger, not a reason to hide a higher-success tradeoff.

If LSTM improves only discovery but lowers arrival, inspect post-detection action changes and physical failures. If the capacity MLP ties LSTM, prefer the simpler model while retaining recurrence as a later tracking candidate. If history stacking ties LSTM, test longer search horizons before accepting recurrent complexity. If the viewpoint or reward experiment wins, address spatial representation or incentives before scaling the network. Combine individually promising changes only in a subsequent matched experiment; individual gains need not add.

The eventual stationary aspiration remains approximately 85% reliable success. Passing 78%/90% is a phase decision gate, not the finished game target. Once stationary seeking qualifies, introduce slow target movement with a fixed schedule and separate reacquisition metrics. A monotonic "cleared forever" map becomes invalid when a target can enter a previously searched region, and indefinitely trusted last-seen coordinates become stale. Add time-dependent search uncertainty and target-motion estimation then, not during this initial sweep.

For browser deployment, a recurrent ONNX model must expose and return its hidden states, with fixtures verifying sequences and resets rather than isolated actions alone. The current deployed known-target brain remains a separate interface. Establish Python success before changing that browser contract. The recommended immediate commitment is the small residual LSTM comparison, not a large memory architecture or a new adversarial game.

Sources

1. **Leaper repository, local primary records.** E:/Leaper/rl_environment.py; train_rl.py; TRAINING.md, sections PPO_27-35; RL_SPEC.md; configurations and evaluation JSONs under E:/Leaper/rl_artifacts/. Hardware/package inspection and TensorBoard timing checked locally on 9 September 2026. Working tree includes uncommitted changes; preserve a snapshot when implementing.
2. **Supplied report.** *Leaper_Memory_Explainer_and_Answers.pdf*. C:/Users/ankud/Downloads/Leaper_Memory_Explainer_and_Answers.pdf, 30 pages, especially phased recommendations, pp. 27-28. Private supplied document; no public URL. The earlier pasted seeker summary is supplementary context.
3. **Leaper paired intervention results.** E:/Leaper/rl_artifacts/note_gate/note_gate_full.json; evaluator E:/Leaper/eval_note_gate.py. Six policies, 100 shared maze seeds, two conditions. Existing results, not new training.
4. **Ni, Tianwei; Benjamin Eysenbach; Ruslan Salakhutdinov.** Recurrent Model-Free RL Can Be a Strong Baseline for Many POMDPs. ICML 2022. Author explanation, CMU, 26 August 2022. Supports the architecture/context/algorithm comparison; does not evaluate Leaper.
5. **Chaplot, Devendra Singh, et al.** Learning To Explore Using Active Neural SLAM. ICLR 2020, author project and paper. Supports modular spatial memory and navigation design.
6. **Yamauchi, Brian.** A Frontier-Based Approach for Autonomous Exploration. IEEE CIRA 1997. Primary paper on known-free/unknown exploration boundaries; used for viewpoint design principles.
7. **Ye, Joel, et al.** Auxiliary Tasks and Exploration Enable ObjectGoal Navigation. ICCV 2021. Author project. Supports testing auxiliary recurrent representation learning.
8. **Ng, Andrew Y.; Daishi Harada; Stuart Russell.** Policy Invariance Under Reward Transformations: Theory and Application to Reward Shaping. ICML 1999. Basis for distinguishing heuristic bonuses from policy-invariant potential shaping.
9. **Andrychowicz, Marcin, et al.** What Matters In On-Policy Reinforcement Learning? A Large-Scale Empirical Study. ICLR 2021; preprint 2020. Supports controlled PPO implementation/activation comparisons.
10. **Baker, Bowen, et al.** Emergent Tool Use From Multi-Agent Autocurricula. Preprint 2019, ICLR 2020. OpenAI research explanation. Different multi-agent setting; cited as future direction, not a local training recipe.
11. **Agarwal, Rishabh, et al.** Deep Reinforcement Learning at the Edge of the Statistical Precipice. NeurIPS 2021. Author project and evaluation tools. Supports uncertainty-aware comparisons and careful aggregation.
12. **Official implementation documentation.** Stable-Baselines3 PPO; sb3-contrib RecurrentPPO; PyTorch installation. Accessed 9 September 2026; local inspection uses SB3/contrib 2.9.0. Supports API, recurrent-state, device, and runtime details.